Engineering pack

The engineering artefacts for MahaSetu AI: requirements targets, API, data and states, security, decisions, operations and testing. Targets are proposals to confirm during the build.

1. Non-functional requirements

These targets make "done" testable. They are proposals for a prototype on one machine; confirm or change them once you have measured.

Area	Requirement	How to verify
Latency	95 percent of read calls (timeline, dashboard) answer within 1 second; submit returns the tracking ID within 2 seconds, with department sends done in the background	Timed test with 50 seeded applications
Reliability	A failed department send retries 3 times with waiting of 1, 2 and 4 seconds, then moves to the dead-letter queue	Failure switch test
Idempotency	A repeated send with the same key creates no duplicate in any department	Automated test, main tab section 10
Consistency	An application's status is always derived from its tasks, never stored separately by hand	Code review plus test
Security	Every route checks role on the server; every cross-department send checks an active, unexpired consent	Automated tests
Auditability	Every login, consent change, send, state change and escalation writes an audit row; the chain verifies	verify command in the demo

Area	Requirement	How to verify
Privacy	Documents are passed as references, not copied; personal fields encrypted at rest; synthetic data only in the prototype	Checklist review
Language	Every label, error and assistant answer exists in Marathi and English	Missing-key check in the build
Accessibility	Status is shown with text and icon, not colour alone; works on a small phone screen	Manual check on a phone
Data retention	Audit rows kept for the life of the project; consent records kept after revoke; test data wiped before any public demo	Written policy
Setup	A new developer starts the whole system from a clean machine with one command	Fresh-machine test
Capacity (prototype)	50 simultaneous users and 1,000 applications without errors	Simple load test

2. API specification

All routes use JSON over HTTPS with a bearer token, except login. Roles are enforced on the server. Turn this table into an OpenAPI file when you build; the FastAPI framework can generate one from your code.

Method and path	Role	Request	Success response	Errors
POST /auth/login	public	email, password	200: token, role, department, language	401 invalid_credentials

Method and path	Role	Request	Success response	Errors
POST /assistant/checklist	citizen, business	service_type, sector, district, size	200: approvals list, each with department, name, reason, rule_id, documents; or an empty list with a "not sure, contact the department" message	422 invalid_input
POST /validate	citizen, business	documents with type and pre-extracted fields	200: ok flag and issues, each with field, code, message in English and Marathi	422 invalid_input
POST /consent	citizen, business	department, purpose, expires_at	201: consent_id	422 invalid_input
GET /consent	citizen, business	none	200: the user's consents	401
DELETE /consent/{id}	citizen, business	none	204; later sends to that department are blocked	404 not_found
POST /applications	citizen, business	service_type, project details, document references; header Idempotency-Key	201: tracking_id and status; the same key again returns the original with 200	403 consent_required, 422 invalid_input

Method and path	Role	Request	Success response	Errors
GET /applications	citizen, business	none	200: the user's applications with derived status	401
GET /applications/{id}/timeline	owner, officer, admin	none	200: events with department, state, time, note	403 forbidden, 404 not_found
POST /applications/{id}/resubmit/{dept}	owner	corrected fields or documents	202: resend to that department only	409 invalid_transition
PATCH /tasks/{id}	officer of that department	state (approved, rejected, more_info) and reason	200: updated task and derived application status	403 wrong_department, 409 invalid_transition
POST /grievances	citizen, business	tracking_id, reason	201: grievance_id	404 not_found
GET /analytics	admin	none	200: average minutes by department, SLA breaches, slowest stages, API health	403 forbidden
GET /admin/failed-calls	admin	none	200: dead-lettered calls	403 forbidden
POST /admin/failed-calls/{id}/retry	admin	none	202: queued for retry	404 not_found
GET /admin/audit	admin	optional tracking_id	200: audit rows	403 forbidden
GET /admin/audit/verify	admin	none	200: ok flag and the first bad row id if any	403 forbidden

Method and path	Role	Request	Success response	Errors
POST /admin/demo/fail-next	admin	department	200	403 forbidden
POST /admin/demo/sla	admin	seconds	200	403 forbidden

Error format. Every error uses one shape, so the frontend can show it in the user's language:

```
{
  "error": {
    "code": "consent_required",
    "message_en": "Please allow data sharing with the Fire department to continue.",
    "message_mr": "पुढे जाण्यासाठी अग्निशमन विभागाला माहिती देण्यास परवानगी द्या."
  }
}
```

Have a native Marathi speaker review every Marathi message before you use it.

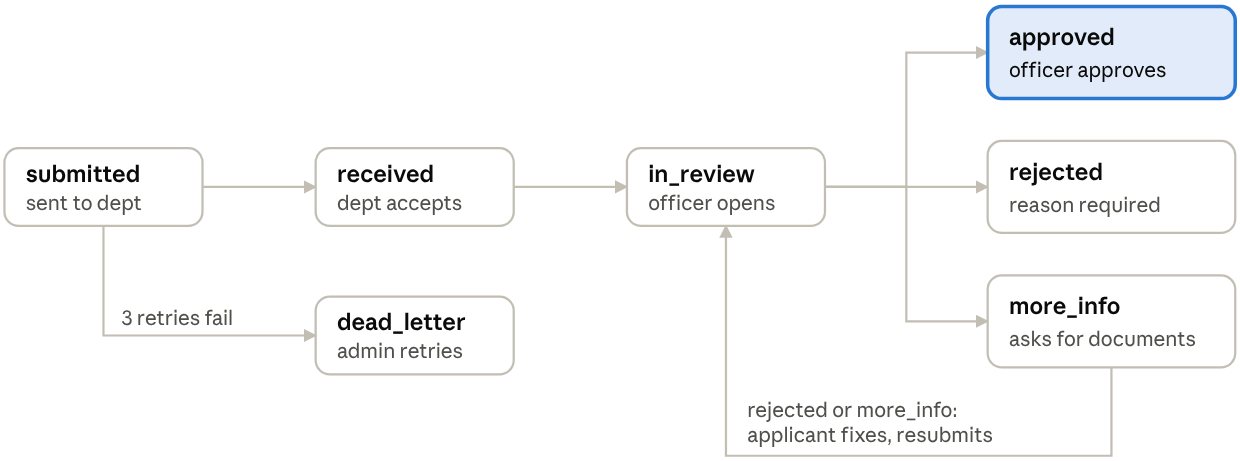
3. Data model and task states

One tracking ID maps to many department tasks. The application's status is computed from its tasks by one function and never edited by hand.

Table	Columns	Keys and indexes
users	id uuid, name text, email text, password_hash text, role (citizen, business, officer, admin), department text (officers only), language (mr, en)	Primary key id; unique email
applications	tracking_id text (format MH-year-number), user_id uuid, service_type text, project jsonb, status text (a cache written only by the status function), created_at timestamptz	Primary key tracking_id; index user_id

Table	Columns	Keys and indexes
dept_tasks	id uuid, tracking_id, dept text, dept_ref_id text, state (see diagram), idempotency_key text, sla_due timestamptz, warned bool, breached bool, created_at, updated_at	Primary key id; unique (tracking_id, dept); unique idempotency_key
documents	id uuid, tracking_id, type text, reference text (storage reference, not a copy), fields jsonb (pre-extracted), validation_status	Primary key id; index tracking_id
consents	id uuid, user_id, dept text, purpose text, granted_at, expires_at, revoked_at (null if active)	Primary key id; index (user_id, dept, purpose)
grievances	id uuid, tracking_id, task_id (optional), reason text, escalation_level int, created_at, resolved_at	Primary key id; index tracking_id
audit_log	id bigserial, actor text, action text, entity text, timestamp, prev_hash char(64), hash char(64)	Primary key id; the app′s database role may insert and read only, never update or delete
dead_letter	id uuid, tracking_id, dept, payload_ref text, error text, attempts int, created_at, retried_at	Primary key id; index tracking_id
api_health	id, connector text, status (ok, error), latency_ms int, last_error text, checked_at	Primary key id; index (connector, checked_at)

A department task ends approved, rejected or waiting for the applicant



department task states · 7 states

A task moves submitted, received, in_review, then ends approved, rejected or more_info. A rejected or more_info task goes back to in_review after the applicant resubmits to that department only. A send that fails three times goes to dead_letter, and an admin retry sends it again.

Allowed transitions only. Any other change is refused with 409 invalid_transition. SLA warned and breached are flags on the task, not states, so a late task keeps its real state.

Derived application status. approved if every task is approved; action_needed if any task is rejected; partly_approved if some are approved and none rejected; otherwise in_progress.

4. Threat model

The prototype uses synthetic data, but design as if the data were real. Trust boundaries: the browser to the backend, the backend to each department connector, and the backend to the language-model service.

#	Threat	Example	Control	How to test
1	Consent bypass	A send goes out with no consent, an expired one or a revoked one	Consent check inside the single send function, with expiry	Automated test: send without consent is refused

#	Threat	Example	Control	How to test
2	Cross-department access	A Fire officer reads a Pollution Control task	Role and department checked on every query, not only in the screen	Test with an officer of another department: 403
3	Audit tampering	Someone edits an audit row after the fact	Hash chain, insert-only database role, verify endpoint	Edit a row, run verify, expect failure
4	Stolen or forged token	A token is copied or made up	Signed short-lived tokens, secret from environment, role re-read from the user record	Expired and tampered token tests
5	Login guessing	Many password attempts	Rate limit and temporary lockout	Repeated-failure test
6	Unsafe upload	A huge or disguised file	Allowed types and size limit, stored outside the web root, never executed	Upload an oversize and a wrong-type file
7	Injection	Crafted input in a form or search	Parameterised queries, input validation	Try quotes and script text in every field
8	Assistant misuse	Prompt injection, or a wrong approval answer	The assistant has no database or tool access, answers only from the rules file and cites the rule, personal data never goes in prompts, "not sure" when no rule matches	Test set including hostile prompts
9	Data leaking through logs	Personal fields printed in an error log	Log identifiers only, never field values	Review logs after a test run

#	Threat	Example	Control	How to test
10	Replay and duplicates	A retry or a repeated click creates two applications	Idempotency key per tracking ID and department	Duplicate-send test
11	Overload	One client floods the API	Rate limits and a cap on queue length	Simple load test
12	Unsafe real integration	Connecting a real department without approval	Mocks only until an MoU and authorised access exist	Process check, not code

Out of scope for the prototype. Penetration testing by a third party, formal compliance review and production key management. Name these as pilot-phase work.

5. Decision records

Five decisions, each with the options considered, so reviewers see the choice was made, not guessed.

#	Decision	Context	Options considered	Chosen and why	Consequence
1	Add an integration layer over existing systems	Departments run separate systems and will not replace them	Replace all systems; build one new portal; integration layer	Integration layer: no replacement, no extra portal, departments keep control	Connectors must be written and maintained per department
2	Assistant is rules-first	A wrong approval answer causes real harm	Free LLM answers; rules only; rules with an LLM to phrase	Rules with an LLM to phrase and handle free text, always citing the rule	Rules file needs an owner; coverage limited to listed services
3	Idempotency key per tracking ID and department	Retries must not create duplicate applications	No protection; deduplicate by content; idempotency key	Key, because it is exact and cheap	Every department adapter must honour the key

#	Decision	Context	Options considered	Chosen and why	Consequence
4	Application status is derived from tasks	Departments finish at different times and can disagree	Store status and update by hand; derive from tasks	Derive through one function, so status cannot drift from the tasks	One partly-approved case needs clear wording in the screens
5	Hash-chained audit log with an insert-only role	Audit evidence must show tampering	Plain log table; hash chain; blockchain	Hash chain with a verify command: tamper-evident without running a blockchain	Writes must be serialised; the chain proves edits, it does not stop a database owner from deleting the whole table

6. Operations and testing strategy

Configuration. Keep every setting in environment variables with a documented .env.example, and never commit real values.

Setting	Purpose
DATABASE_URL, REDIS_URL	Connections to PostgreSQL and Redis
JWT_SECRET, TOKEN_MINUTES	Token signing key and lifetime
LLM_API_KEY, LLM_ENABLED	Assistant service; set LLM_ENABLED to false to use rules only
SLA_FILE, RULES_FILE	Paths to the SLA durations and approvals rules files
DEMO_MODE	Turns the demo controls on; must be off outside demos

Run, check, back up.

- Start everything with one command (`docker compose up --build`) and confirm with the /health route and the api_health table.
- Log one line per request with request id, route, status and time; never log field values.

- Watch three signals: failed-call count in the dead-letter queue, SLA breaches per department, and connector latency.
- Back up the database with a scheduled dump, and practise restoring it once.
- Tag each release, and keep a short changelog.

Testing strategy

Level	What it proves	Target
Unit	Each connector maps every field to the common model and back; status function; rule lookups	All mapped fields covered; all status cases covered
Contract	Every connector passes the same tests against its mock, so a new department is "write one adapter, pass the tests"	One shared test file run per connector
Integration	Consent gate, idempotency, retry then dead-letter, hash-chain verify, role checks	All pass from a clean start
End to end	The full demo story from section 11 of the main tab, including one forced failure and one SLA breach	Runs in one command and prints timings
Assistant evaluation	Marathi and English answers against a set of 30 to 50 questions with known answers, including hostile prompts	Record accuracy and how often it correctly says it is not sure
Security	The tests listed in the threat model	All pass
Load	50 simultaneous users and 1,000 applications	No errors; record the slowest calls
Usability	Five real users finish an application, including a Marathi-first user and a low-digital-literacy user	Record completion, time and where they got stuck, plus one change you made

Keep every failure you find and its fix in the evidence folder. One of them becomes the honest failure on your project page.

7. Glossary and open questions

Term	Meaning
Canonical data model	One agreed shape for an application, so each department's format is translated to and from it
Connector	A small adapter that translates between one department's format and the canonical model
Gateway	The single door through which every call to a department passes
Idempotency key	A unique label on a send, so a repeat of the same send is recognised and not duplicated
Dead-letter queue	Where a call goes after repeated failure, so an admin can inspect and retry it
SLA	The deadline a department has to act on a task
Consent	The user's recorded permission to share named data with a named department for a named purpose
Tracking ID	The one number a user follows, mapped to one task in each department
Hash chain	A log where each row stores a fingerprint of the previous row, so an edit breaks the chain
Mock department	A pretend department system with synthetic data, used for the prototype

Open questions to settle

- [ ] Which three or four real services will the pilot cover first, and who owns each rules entry?
- [ ] Which department will host the pilot, and who is the nodal officer?
- [ ] Which identity and document services (for example DigiLocker through API Setu) can the pilot use, and under what access terms? Confirm with the department rather than assuming.
- [ ] What retention period does the state require for audit and consent records?

- [ ] Who reviews and approves the Marathi wording, and how often?
- [ ] What SLA durations does each department actually commit to?
- [ ] Where will the pilot be hosted, given a preference for in-state hosting?